

Exactly 😊 — **this is the missing piece**. The distinction is determined by **how the function is declared**, not by what the function happens to do.

1. Ordinary function vs Server Function

Ordinary function

```
function saveProduct(product) {  
  console.log(product);  
}
```

This is just a normal JavaScript function.

It has **no Server Function semantics**.

```
function saveProduct(...)  
  ↓  
ordinary JS function  
  ↓  
✗ not a Server Reference
```



Server Function

A function becomes a Server Function when it is explicitly marked with:

```
'use server';
```

For example:

```
'use server';

export async function saveProduct(product) {
  await db.products.save(product);
}
```

Now React/Next.js knows:

```
saveProduct
  ↓
Server Function
  ↓
Server Reference
```



So the browser can invoke it through the Server Action mechanism rather than receiving the function implementation.

2. There are TWO places 'use server' can appear

This is important.

A. Module-level

```
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The module is server-only and its exported functions are Server Functions.

B. Inline inside a function

You can also have:

```
export default function Page() {  
  
  async function addToCart(productId) {  
    'use server';  
  
    await db.cart.add(productId);  
  }  
  
  return (  
    <form action={addToCart}>  
      ...  
    </form>  
  );  
}
```

Here:

```
addToCart()  
  ↓  
'use server'  
  ↓  
Server Function
```



The `'use server'` directive is what gives it that special meaning.

3. Compare these side-by-side

✗ Ordinary function

```
function handleClick() {  
  console.log("clicked");  
}
```

```
ordinary JavaScript function  
  ↓  
no Server Reference
```



✓ Server Function

```
async function addToCart() {  
  'use server';  
  
  await db.cart.add();  
}
```

Server Function

↓

Server Reference

↓

can be invoked remotely



4. But here's a VERY important distinction

Don't confuse:

```
'use client';
```

with:

```
'use server';
```

They establish very different things.

`'use client'`

Marks a **Client Component/module boundary**:

```
'use client'
```

↓

Client Component graph

↓

browser JS



`'use server'`

Marks a **Server Function**:

```
'use server'
  ↓
Server Function
  ↓
Server Reference
  ↓
remote invocation
```



5. What happens with `onClick`?

Suppose:

```
'use client';

function Button() {

  function handleClick() {
    console.log("clicked");
  }

  return (
    <button onClick={handleClick}>
      Click
    </button>
  );
}
```

`handleClick` is an **ordinary function**.

But that's completely fine because it was created **inside the Client Component**.

```
Client Component
  |
  ↓
handleClick()
  |
  ↓
Browser
```



It doesn't need to cross the Server → Client boundary.

6. Now compare with a Server Function

```
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

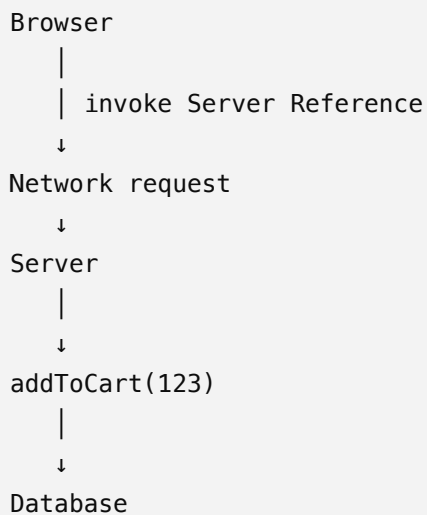
Then:

```
'use client';

import { addToCart } from './actions';

function AddButton() {
  return (
    <button onClick={() => addToCart(123)}>
      Add
    </button>
  );
}
```

Conceptually:



The browser **doesn't** receive the implementation of:

```
async function addToCart() {
  // database access
}
```

7. One subtle correction: "Server Function" ≠ "any function that runs on server"

This is **really important**.

Suppose you have:

```
export default async function Page() {  
  
  const data = await fetchData();  
  
  return <div>{data.name}</div>;  
}
```

`fetchData()` may execute on the server.

But that does **not automatically make it a Server Function**.

It's simply a server-side function.

```
fetchData()  
  ↓  
runs on server  
  ↓  
ordinary server-side function
```



It isn't remotely callable by the browser.

Compare:

```
Server-side function  
  ≠  
Server Function
```



A **Server Function** has the explicit Server Function semantics (`'use server'`).

8. This distinction is GOLD for interviews 🎯

Imagine the interviewer asks:

"Are all functions in a Server Component Server Actions?"

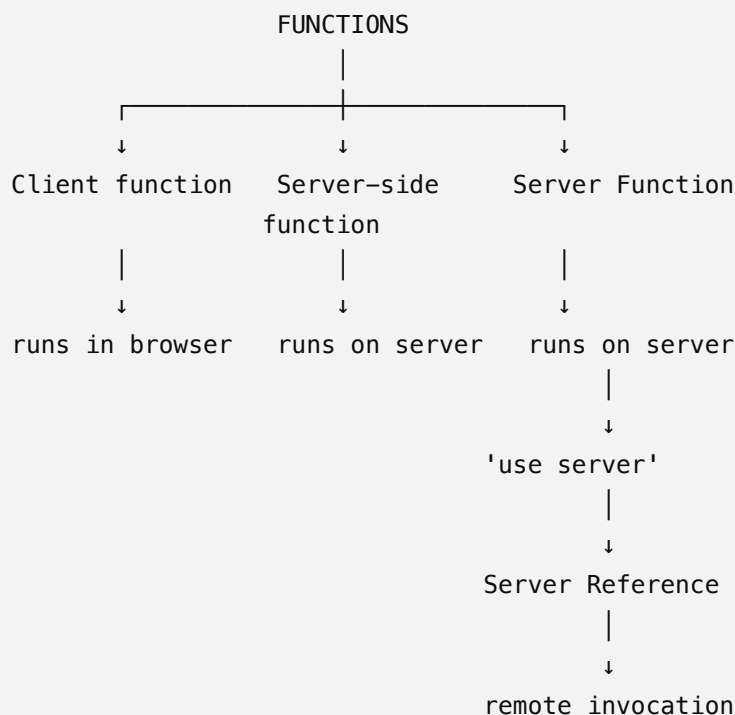
Answer:

"No. A function merely executing on the server isn't automatically a Server Function. A Server Function is explicitly marked with `'use server'`, which gives it Server Reference semantics so it can be invoked from the client through the Server Action mechanism."

That's a very strong answer.

9. Your mental model

Keep these three categories separate:



And one final rule:

"Where the function runs" and "whether it is a Server Function" are two different questions.

That's the distinction that usually trips people up. 🤔



okay next

Absolutely 🤔 Let's move to the next piece.



6.11.3 — Client References & Server References Under the Hood

We now know:

Ordinary function
≠
Server Function



and:

'use server'
↓
Server Function
↓
Server Reference



Now let's understand **what these "references" actually are**, because they are a key part of how Flight connects the two worlds.

1. First, forget functions for a moment

Think about a normal JavaScript module:

```
</> TypeScript  
  
import ProductList from './ProductList';
```



At build time, Next.js knows:

ProductList
↓
./ProductList
↓
client module



When `ProductList` is a Client Component:

```
</> TypeScript  
  
'use client';  
  
export default function ProductList() {  
  return <div>Products</div>;  
}
```



Next.js needs to make that module available to the browser.

So the build system creates a **client module graph/bundle**.

Conceptually:

```
ProductList.tsx
  ↓
Next.js bundler
  ↓
Client JS module
  ↓
browser
```



2. What does the Server Component tree need to say?

Suppose:

```
</> TypeScript

export default function Page() {
  return (
    <main>
      <h1>Products</h1>
      <ProductList />
    </main>
  );
}
```



React on the server encounters:

```
<ProductList />
```



But `ProductList` is a Client Component.

React therefore needs a way to represent:

"At this position in the tree, use that client-side module."

It doesn't send the component's source code through the RSC payload.

Instead, it can represent a **Client Reference**.

Conceptually:

```
Client Reference
{
  module: ProductList,
  export: default
}
```



⚠ This is conceptual—not the literal Flight object.

3. So what is a Client Reference?

A useful definition:

A Client Reference is metadata that allows the RSC system to refer to a Client Component/module from the server-rendered component tree.

Think:

```
SERVER
|
| "I need Client Component X here."
↓
Client Reference
|
↓
CLIENT
|
↓
Client module X
```



The reference is the **bridge**.

4. Why not just send the JavaScript?

Because that would defeat one of the major benefits of Server Components.

Imagine:

```
</> TypeScript

export default async function Page() {
  const hugeData = await databaseQuery();

  return <ProductList data={hugeData} />;
}
```



The server can execute the Server Component.

The browser doesn't need:

```
Page()  
databaseQuery()  
database credentials  
server-only logic
```



The browser only needs:

```
Client Component code  
+  
serialized data needed by it
```



So the boundary becomes:

SERVER



```
Server Component code  
Database  
Secrets  
Server logic
```

```
|  
| RSC Payload  
↓
```

CLIENT

```
Client Component code  
Client React  
Browser APIs
```

5. Now let's reverse the direction

Suppose we have:

```
</> TypeScript
```



```
'use server';  
  
export async function deleteProduct(id) {  
  await db.products.delete(id);  
}
```

The browser needs to be able to say:

```
"Run deleteProduct with id=123"
```



But it doesn't need:

```
</> JavaScript
```



```
async function deleteProduct(id) {  
  await db.products.delete(id);  
}
```

So we need the reverse bridge:

```
Server Function
```



```
Server Reference
```



```
Browser
```



6. What does a Server Reference represent?

A useful definition:

A Server Reference is a reference/identifier that lets the client request invocation of a specific Server Function without transferring the function's implementation to the browser.

Conceptually:

```
Server
```



```
deleteProduct()
```



```
Reference ID: XYZ
```

Then:

```
Browser
```



```
XYZ(123)
```



```
Server
```



↓
deleteProduct(123)

Again, the exact wire protocol is more complicated than this simplified picture.

7. Now look at the symmetry

This is worth remembering forever. 😊

Server → Client

Server Component
↓
Client Reference
↓
Client Component

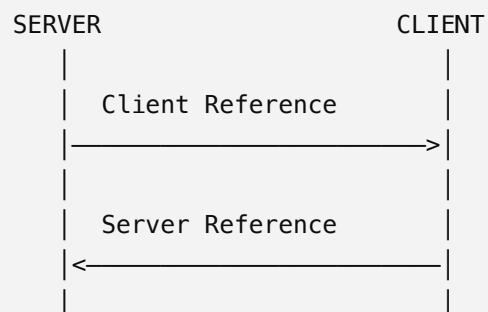


Client → Server

Client interaction
↓
Server Reference
↓
Server Function



So:



References are the bridges between the two execution environments.

8. How does this relate to the RSC Payload?

Now we're getting to the important part.

The RSC payload can contain references.

Conceptually:

RSC PAYLOAD

Record 1

→ main

Record 2

→ "Products"

Record 3

→ Client Reference

→ ProductList

Record 4

→ props

→ products



So when React receives the payload:

RSC Payload

```
|
├─ tree information
├─ serialized values
└─ Client References
```



React can reconstruct the relationship:

main

```
├─ h1
│   └─ Products
└─ ProductList
```



with:

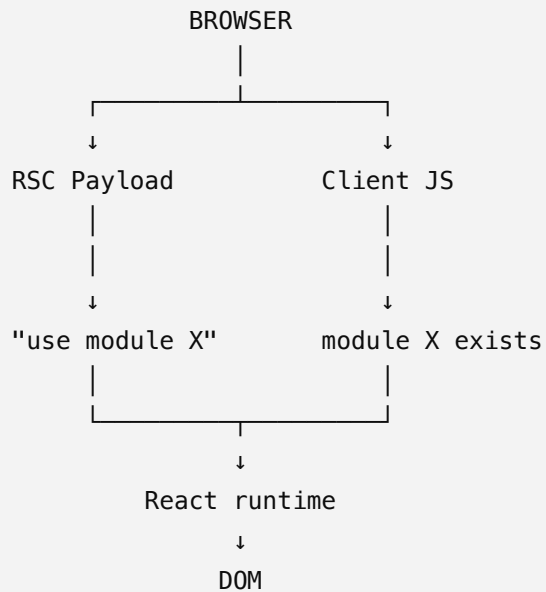
ProductList

↓
client module



9. What about the Client JS?

This is where the two pieces meet.

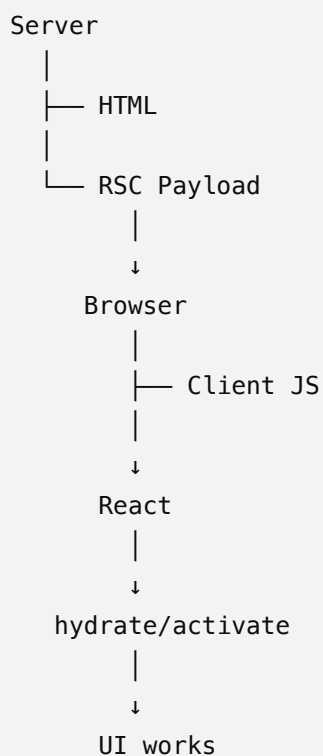


So the RSC payload says **what/reference**, while the client bundle provides **the executable client code**.

10. Why this matters for hydration

Now let's connect it to something we already learned.

Initial load:



The browser gets the initial visual HTML.

Then React uses the RSC information + Client Component code to establish the client-side React tree and make the interactive parts work.

11. And during navigation?

Now you're on:

/products



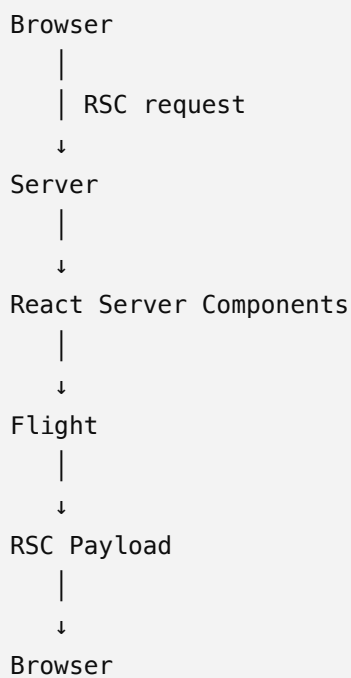
You click:

`</> TypeScript`



`<Link href="/products/123">`

The client router requests the new RSC result.



The payload may contain:

new tree
+
serialized values
+
Client References



Then React updates the existing tree.

12. This is why Router Cache stores RSC information

Now you can finally see why we kept saying:

Router Cache stores RSC payloads/segments.

Because the cache isn't primarily storing:

HTML files



Instead, conceptually:

Router Cache



```
graph TD
    RC[Router Cache] --- P1[ /products ]
    P1 --- R1[ RSC tree/data ]
    RC --- P2[ /products/123 ]
    P2 --- R2[ RSC tree/data ]
    RC --- P3[ /about ]
    P3 --- R3[ RSC tree/data ]
```

When navigation happens:

Click

↓

Router Cache?

↓

YES

↓

use RSC data



or:

Click

↓

Router Cache?

↓

NO

↓

request RSC payload

↓

store it

↓

React



13. One subtle point: References aren't URLs

Don't think:

```
Client Reference
=
URL
```



or:

```
Server Reference
=
API endpoint
```



They're **React/framework-level references**.

The actual network protocol determines how those references are encoded and invoked.

Conceptually:

```
Client Reference
≈ "this client module/export"
```

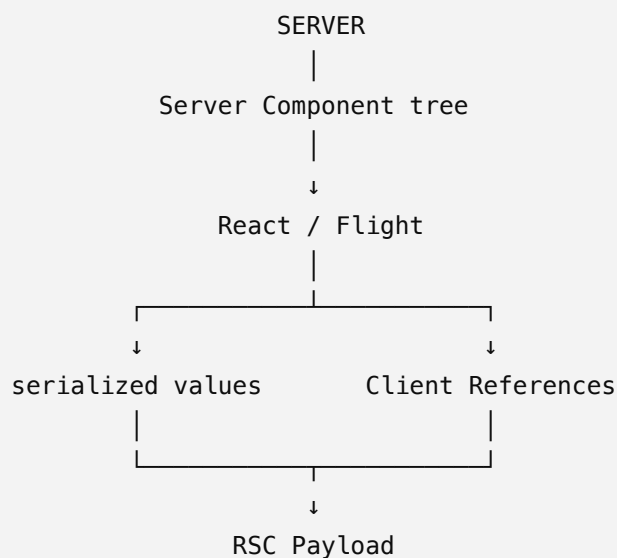


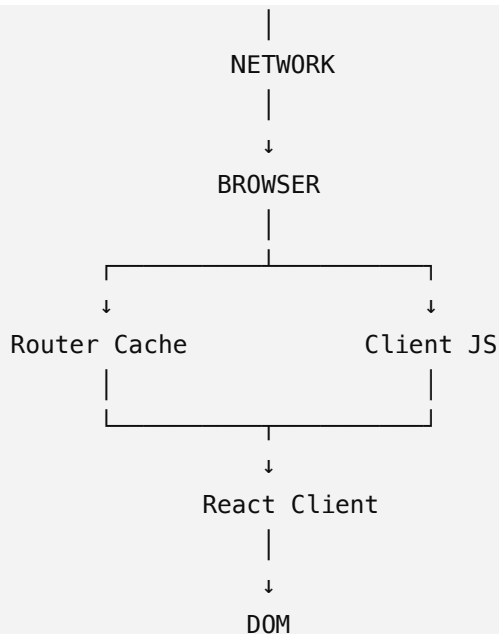
```
Server Reference
≈ "this server function"
```

They're identifiers/handles, not simply HTTP URLs.

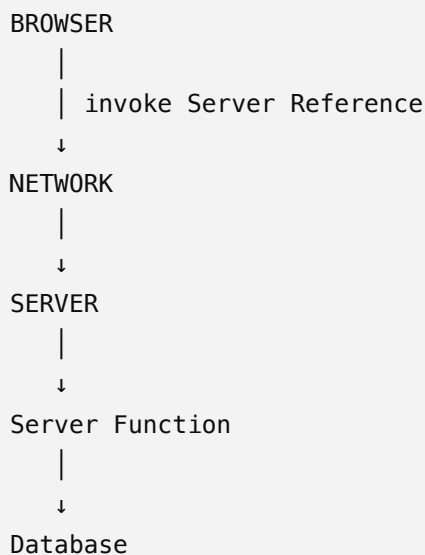
14. The complete architecture 🧠

Now put the last three lessons together:





For server mutations:



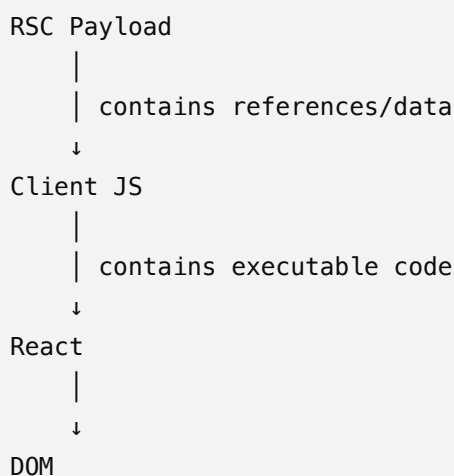
What I want you to know before moving on

At this point, you should be able to distinguish:

Concept	Meaning
Flight	React's protocol/mechanism for RSC communication
RSC Payload	Flight-formatted data for a particular render/navigation
Serialization	Converting the component result/data/references into transferable representation

Concept	Meaning
Client Reference	Reference to a Client Component/module
Client JS	Actual executable Client Component code
Server Function	Function explicitly given Server Function semantics
Server Reference	Reference allowing client to invoke that Server Function
Router Cache	Client-side cache of RSC route/segment information

And the most important distinction:



Next up: 6.11.4 — Let's actually inspect a real RSC request. 🔍

We'll take the theory into **Chrome DevTools → Network**, identify the RSC request, look at the request headers, response, and connect each thing you see to:

Flight → RSC Payload → Client Reference → Router Cache

That's the point where this stops being abstract and you can literally **see the machinery happening on your laptop.** 😊



😊 Perfect. Next we'll do **6.11.7 — the complete RSC + React Flight recap and interview questions.**

After that, **6.11.x is officially done 🏁**, and we'll start **6.12 in a new chat.**



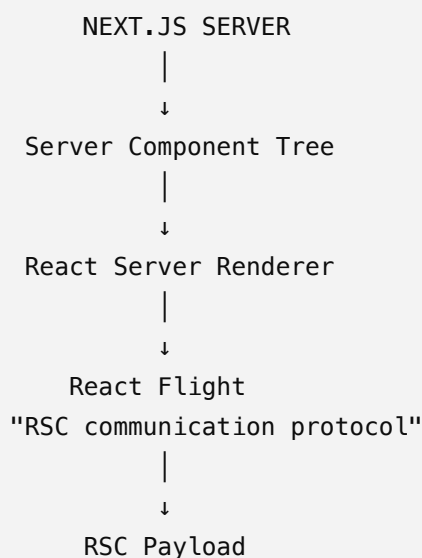
sure go ahead

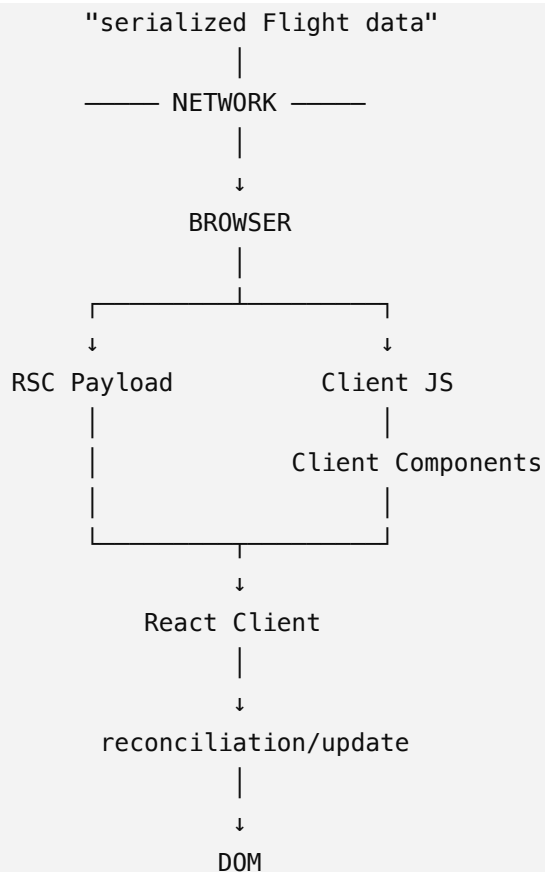
🏁 6.11.7 — Complete RSC Payload + React Flight Recap

Alright 😊 this is the **final 6.11.x lesson**. After this, we're done with RSC/Flight fundamentals and **6.12 will start in a fresh chat**, as planned.

Let's consolidate everything into one mental model first.

1. The entire RSC story in one picture





The biggest thing to remember:

Server Components don't need to be shipped to the browser as JavaScript. Their rendered result is represented in the RSC payload.

Client Components are different because their JavaScript needs to be available in the browser for interactivity.

2. Flight vs RSC Payload

This was one of your original questions.

React Flight

Think:

The protocol/mechanism used by React to transport Server Component information.

RSC Payload

Think:

The actual serialized data produced using that mechanism for a particular render/navigation.

So:

Flight

↓

protocol / mechanism

RSC Payload

↓

actual Flight data



Don't say:

"Flight is the payload."



Say:

"The RSC payload is transmitted using React Flight."



3. What does "serialized representation" mean?

Suppose the server has:

`</> TypeScript`

```
<Product
  name="Dosa"
  price={80}
/>
```



React has an internal representation of this component tree.

That internal representation can't simply be sent directly over HTTP.

So React converts the relevant information into a **transferable Flight representation**.

React tree

↓

serialization

↓

Flight representation

↓

network



↓
client React

So:

Serialized representation = a network-transferable representation of the Server Component tree, supported values, and relevant references.

And importantly:

Flight serialization
≠
JSON.stringify()



Flight has React-specific semantics.

4. Client References

Suppose:

```
</> TypeScript
'use client';

export default function AddButton() {
  return <button>Add</button>;
}
```



The Server Component tree can contain:

```
</> TypeScript
<AddButton />
```



The server doesn't need to send the component's source code inside the RSC payload.

Instead, Flight can represent a **Client Reference**.

Conceptually:

```
Server
|
| "Client Component X goes here"
↓
Client Reference
|
↓
```



```
Browser
|
↓
client JS module
|
↓
AddButton
```

So:

```
Client Reference
  ≠
Client Component source code
```

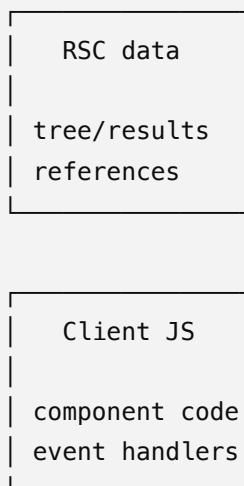


It's a reference to the client-side module/export.

5. Client JS vs RSC Payload

Don't mix these up.

BROWSER



The two work together.

RSC Payload

Answers roughly:

"What Server Component result/tree should React use?"

Client JS

Provides:

"Here is the executable code for Client Components."

6. Server Functions / Server References

Now reverse the direction.

```
</> TypeScript

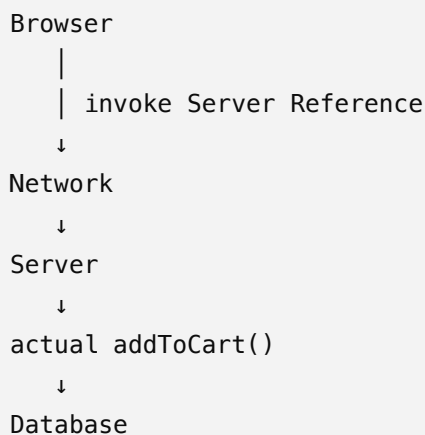
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The browser doesn't receive the implementation.

Instead, it can receive/use a **Server Reference**.

Conceptually:



And remember the important distinction:

```
function that happens to execute on server
    ≠
Server Function
```

A Server Function has explicit Server Function semantics, typically via:

```
</> TypeScript

'use server';
```

7. Ordinary function vs Server Function

Ordinary server-side function

</> TypeScript



```
function getProducts() {  
  return db.products.findMany();  
}
```

It may execute on the server.

But it isn't automatically remotely invocable.

server-side function

↓

runs on server

↓

ordinary function



Server Function

</> TypeScript



```
'use server';  
  
export async function addToCart() {  
  ...  
}
```

Server Function

↓

Server Reference

↓

can be invoked through supported server-action mechanisms



8. Serialization boundary

Suppose:

</> TypeScript



```
export default function Page() {  
  const product = {  
    id: 1,  
  }  
}
```

```

    name: "Dosa"
  };

  return <ProductCard product={product} />;
}

```

and:

```

</> TypeScript

'use client';

function ProductCard({ product }) {
  ...
}

```

The `product` data crosses:

```

Server
  |
  | serializable value
  ↓
Client boundary
  |
  ↓
Client Component

```

But an ordinary server function cannot simply be passed the same way:

```

</> TypeScript

<ProductCard
  onSave={someServerFunction}
/>

```

unless that function has the appropriate **Server Function/reference semantics**.

9. 'use client' vs 'use server'

Keep this distinction very clear.

'use client'

```

'use client'
  ↓
Client Component boundary
  ↓

```

```
client module graph
```

```
↓
```

```
browser JS
```

```
'use server'
```

```
'use server'
```

```
↓
```

```
Server Function semantics
```

```
↓
```

```
Server Reference
```

```
↓
```

```
remote invocation
```



And:

`'use client'` does **not** simply mean "this component only renders in the browser."

That's a common oversimplification.

10. RSC navigation

Now suppose:

```
/products
```



and:

```
<> TypeScript
```

```
<Link href="/products/123">
```



is clicked.

The important flow is:

```
<Link>
```

```
↓
```

```
Next.js Client Router
```

```
↓
```

```
RSC request
```

```
↓
```

```
Server Components execute/render
```

```
↓
```

```
React Flight
```

```
↓
```



```
RSC Payload
↓
Browser
↓
React reconciliation
↓
DOM update
```

Notice:

No full HTML document replacement is required for this client-side navigation.

11. DOM can change without replacing the document

This was another question you asked earlier.

Suppose:

Before

```
<main>
  <h1>Products</h1>
  <ProductList />
</main>
```



After navigation

```
<main>
  <h1>Product 123</h1>
  <ProductDetails />
</main>
```



The DOM can change substantially.

But:

```
DOM changed
  ≠
whole HTML document replaced
```



React can update the existing DOM based on the new RSC result.

That's a **very important App Router concept**.

12. Router Cache

The RSC payload received during navigation can be used by the App Router and stored in its client-side Router Cache.

Conceptually:

```
Server
  ↓
RSC Payload
  ↓
Router Cache
  ↓
React
  ↓
DOM
```



Router Cache is:

Client-side cached RSC route/segment information used by the App Router.

Don't confuse it with:

```
Data Cache
Full Route Cache
browser HTTP cache
JS bundle cache
```



They're different things.

13. Cache HIT vs MISS

HIT

```
Navigation
  ↓
Router Cache
  ↓
HIT ✓
  ↓
reuse cached RSC information
  ↓
React
  ↓
DOM
```



MISS



```
graph TD; Navigation --> RouterCache1[Router Cache]; RouterCache1 --> MISS[MISS ❌]; MISS --> RSCrequest[RSC request]; RSCrequest --> Server; Server --> Flight; Flight --> RSCPayload[RSC Payload]; RSCPayload --> RouterCache2[Router Cache]; RouterCache2 --> React;
```

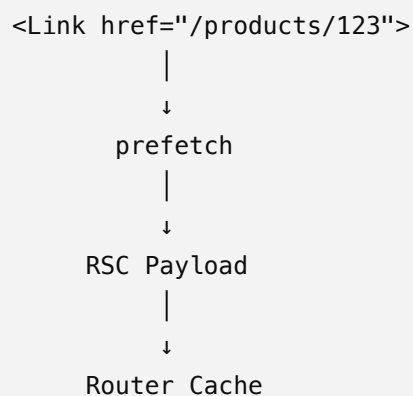
Navigation
↓
Router Cache
↓
MISS ❌
↓
RSC request
↓
Server
↓
Flight
↓
RSC Payload
↓
Router Cache
↓
React

This is the core navigation-caching model.

14. Prefetching

A `<Link>` can also be prefetched in applicable situations.

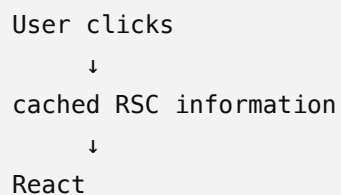
Conceptually:



```
graph TD; Link["<Link href='/products/123'>"] --> prefetch; prefetch --> RSCPayload[RSC Payload]; RSCPayload --> RouterCache[Router Cache];
```

`<Link href="/products/123">`
↓
prefetch
↓
RSC Payload
↓
Router Cache

Then:



```
graph TD; UserClicks[User clicks] --> cachedRSC[cached RSC information]; cachedRSC --> React;
```

User clicks
↓
cached RSC information
↓
React

↓
UI

So a request can happen **before the user actually navigates**.

15. Streaming + Suspense

Now connect the last part.

Suppose:

Header → fast
Products → fast
Reviews → slow



With:

`</> TypeScript`



```
<Suspense fallback={<ReviewsSkeleton />}>  
  <Reviews />  
</Suspense>
```

React can conceptually produce:

Header
↓
Products
↓
ReviewsSkeleton



then later:

Reviews
↓
replace fallback



Flight allows the RSC result to be **streamed progressively**.

So:

Suspense
↓
defines boundary

Streaming



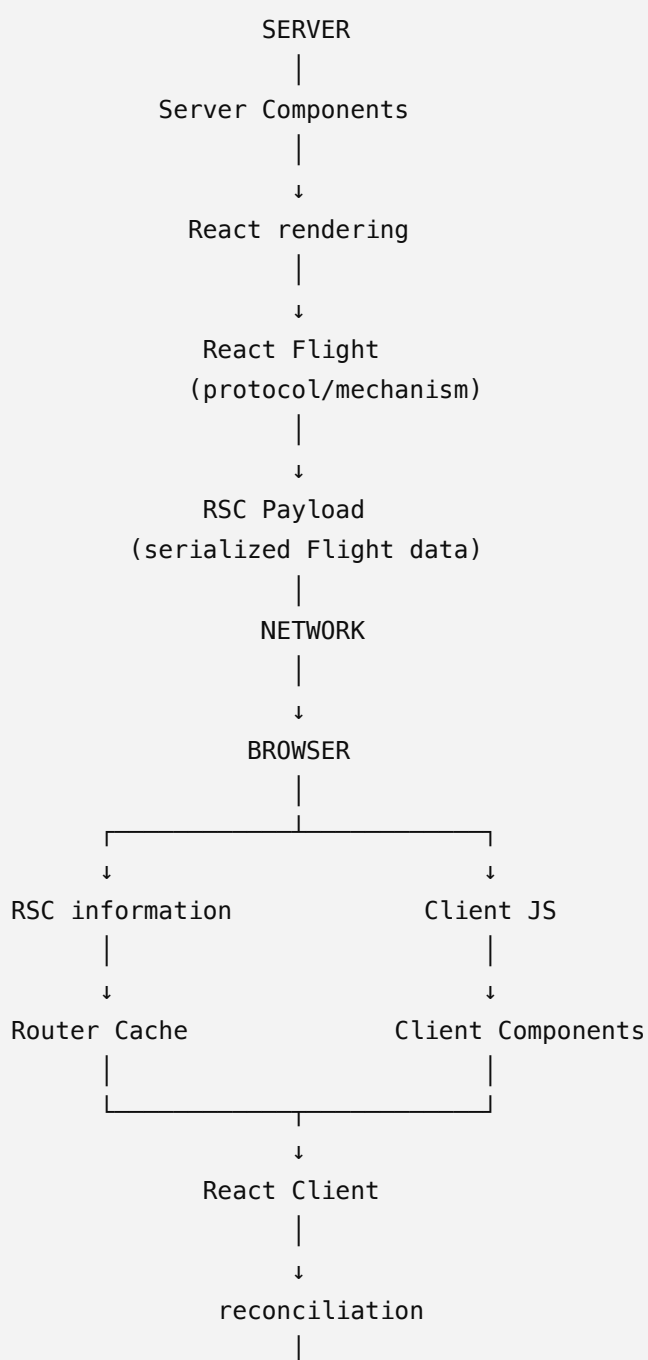
↓
delivers results progressively

Flight
↓
transports RSC representation

These aren't interchangeable concepts.

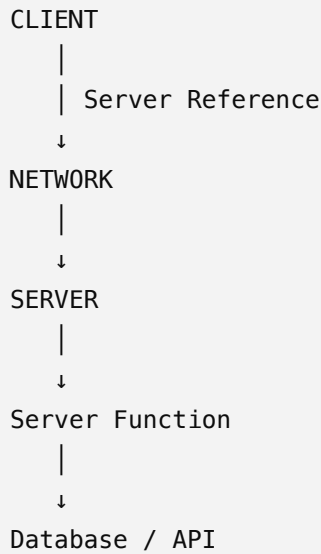
16. The complete architecture 🔥

Here's your final 6.11 mental model:



↓
DOM

For server mutations:



🎯 17. Interview Questions

Now let's test whether you've actually got it rather than just nodding along. 😂

Q1. What is React Flight?

Answer:

React Flight is the protocol/mechanism React uses to communicate the Server Component tree between server and client.

Q2. What is the RSC Payload?

The RSC Payload is the serialized Flight data representing the Server Component result/tree, supported values, and relevant references for a particular render or navigation.

Q3. Is RSC Payload HTML?

No.

HTML

→ document/UI representation



RSC Payload

→ React Server Component representation

Q4. Is RSC Payload JSON?

Not simply.

Flight has its own serialization format and semantics beyond ordinary JSON serialization.

Q5. Does the browser receive Server Component source code?

No.

The Server Component executes on the server, and its result is represented in the RSC payload.

Q6. Does the browser receive Client Component JavaScript?

Yes.

☐ Client Components need client-side JavaScript for their interactivity.



Q7. What is a Client Reference?

A reference in the RSC system to a Client Component/module that the browser can resolve to its client-side code.

Q8. What is a Server Reference?

A reference to a Server Function that allows the client to request its invocation without receiving the function implementation.

Q9. Can any function become a Server Function just because it runs on the server?

No.

```
runs on server
  ≠
Server Function
```



Server Function semantics are explicitly established, typically with `'use server'`.

Q10. During `<Link>` navigation, does Next.js normally download a complete new HTML document?

No.

For client-side App Router navigation, Next.js can request the RSC payload and use it to update the existing React tree and DOM.

Q11. Can the DOM structure change during client navigation?

Absolutely.

```
existing DOM
  ↓
React receives new RSC result
  ↓
reconciliation
  ↓
new DOM structure
```



But the entire browser document doesn't have to be replaced.

Q12. What does Router Cache store?

Client-side RSC route/segment information used by the App Router for navigation and reuse.

18. Your 30-second interview answer

If they ask:

"Explain React Server Components and RSC Payload in Next.js."

You can say:

"Next.js uses React Server Components to execute server-side component logic. React's Flight protocol provides the mechanism for representing and transporting the Server Component result to the client as an RSC payload. The payload contains serialized component-tree information, supported values, and references to Client Components. The client combines that information with the Client Component JavaScript and React uses it to construct or reconcile the UI. During App Router navigation, the RSC payload can be fetched and stored in the client Router Cache, allowing the existing application tree to be updated without a full document reload."

That's a **very solid mid-level answer**. 🎯

🏁 6.11.x — COMPLETE

You now have:

6.11 RSC Payload + React Flight



- |
- |— Flight
- |— RSC Payload
- |— serialization
- |— Client References
- |— Server References
- |— Server Functions
- |— RSC navigation
- |— Router Cache relationship
- |— prefetching
- |— streaming
- |— Suspense

🏁 DONE

And now we **STOP** this chat. 🙅

As planned, **don't start 6.12 here**.

Open a **new chat** and say:

"Continue Next.js 6.12"

Then we'll begin the next major piece:

6.12 — Serialization, Client References & Server References

We'll go deeper there, without mixing it back into 6.11. 😊



server function vs server side function invocation and diff

Yes — this distinction is **super important**, because "server-side function" and "Server Function" sound almost identical. 😊

1. Server-side function

A **server-side function** simply means:

A normal function whose code executes on the server.

Example:

```
</> TypeScript

async function getProducts() {
  const products = await db.products.findMany();
  return products;
}

export default async function Page() {
  const products = await getProducts();

  return <ProductList products={products} />;
}
```

Here:

```
Page()
  ↓
getProducts()
  ↓
Database
```

`getProducts()` runs on the server.

But it's just a **normal function call**:

```
Page()  
  |  
  | normal JS call  
  ↓  
getProducts()
```



There is **no network request** between **Page** and **getProducts**.

The function isn't remotely callable by the browser.

2. Server Function

A **Server Function** is different.

It is explicitly given Server Function semantics, typically with:

```
</> TypeScript  
  
'use server';  
  
export async function addToCart(productId) {  
  await db.cart.add(productId);  
}
```



Now the function can be exposed through a **Server Reference**.

Conceptually:

```
Browser  
  |  
  | invoke Server Reference  
  ↓  
NETWORK REQUEST  
  ↓  
Server  
  ↓  
addToCart(123)
```



So there's a fundamental difference:

	Server-side function	Server Function
Runs on server	✓	✓

	Server-side function	Server Function
Normal JS function	✓	Also a function, but special semantics
'use server'	✗	✓
Callable remotely from client	✗	✓ through supported mechanisms
Server Reference	✗	✓
Network involved when called from server code	✗	Not necessarily
Designed for client → server invocation	✗	✓

3. "Server-side function invocation" means something different

Suppose:

```
</> TypeScript

async function getUser() {
  return db.user.find();
}

async function Page() {
  const user = await getUser();
}
```

The invocation is:

```
Page()
|
| function call
↓
getUser()
```

Everything is happening inside the **same server execution environment**.

There's no:

```
browser
  ↓
network
  ↓
server
```



It's simply:

```
Server
|
├── Page()
|   |
|   └── getUser()
|
└── DB
```



4. Server Function invocation is different

Now:

```
</> TypeScript

'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```



And the browser wants to invoke it.

The browser doesn't have:

```
</> JavaScript

addToCart() // implementation
```



Instead it has a reference.

Conceptually:

```
Client
|
| Server Reference: XYZ
| arguments: 123
↓
Network
```



```
↓  
↓  
Server  
|  
| resolve XYZ  
↓  
addToCart(123)
```

That's a **remote invocation**.

5. Here's the easiest way to remember it 🧠

Ask:

"Who is calling the function?"

If it's:

```
Server code  
↓  
Server function
```



that's just a normal function invocation, even if the function is a Server Function.

No special network trip is needed.

But if:

```
Browser  
↓  
Server Function
```



then the browser can't directly execute it.

It invokes the **Server Reference**, which causes communication with the server.

6. This is the subtle point

A Server Function doesn't mean:

"Every time this function is called, a network request happens."



It means:

The function has semantics that allow it to be referenced and invoked from the client.

So:

```
</> TypeScript

'use server';

export async function addToCart(id) {
  ...
}
```

If server code does:

```
</> TypeScript

await addToCart(123);
```

that's simply:

```
Server
  ↓
normal function invocation
  ↓
addToCart()
```

But when the **client** invokes it through its **Server Reference**, then:

```
Client
  ↓
Server Reference
  ↓
network
  ↓
Server Function
```

7. One more example

Normal server-side helper

```
</> TypeScript

async function calculatePrice(productId) {
  const product = await db.products.find(productId);
```

```
    return product.price * 1.18;
  }
```

Used by:

⌵ TypeScript



```
export default async function Page() {
  const price = await calculatePrice(123);

  return <div>{price}</div>;
}
```

Flow:

Page()

↓

calculatePrice()

↓

DB



No client involvement.

Server Function

⌵ TypeScript



```
'use server';

export async function purchase(productId) {
  await db.orders.create({ productId });
}
```

Client:

⌵ TypeScript



```
'use client';

import { purchase } from './actions';

export function BuyButton({ productId }) {
  return (
    <button onClick={() => purchase(productId)}>
      Buy
    </button>
  );
}
```

```
);  
}
```

Conceptually:

```
CLICK  
↓  
purchase(123)  
↓  
Server Reference  
↓  
NETWORK  
↓  
Server  
↓  
actual purchase(123)  
↓  
DB
```



The interview distinction

If they ask:

"What's the difference between a server-side function and a Server Function?"

Say:

"A server-side function is simply a normal function that executes in the server environment. It can be called by other server code through an ordinary JavaScript function call. A Server Function has explicit Server Function semantics, typically through `'use server'`, which allows it to be represented by a Server Reference and invoked from client code through the server-action mechanism. The function implementation remains on the server."

And the one-liner:

```
Server-side function  
= where it executes
```

```
Server Function  
= special semantics for server invocation from the client
```



 That distinction is exactly what I want you to carry into 6.12.



ChatGPT can make mistakes. Check important info.